

Recurrent Neural Networks

Why they are needed, the vanilla RNN, activations, parameter counting, RNN types, stacking, and the math of sequence generation — step by step

Moin Mostakim

Outline

- 1 Why we need RNNs
- 2 The vanilla RNN
- 3 Activation: why tanh
- 4 Counting parameters
- 5 RNN types and stacking
- 6 The math of sequence generation
- 7 Applications and summary

The limitation of feedforward networks

- A multilayer perceptron (MLP) maps a **fixed-size** vector to an output: $\mathbf{y} = f(\mathbf{x})$.
- It has **no memory**: each input is processed independently, so it cannot use what it saw earlier.
- Real data is often **sequential** and of **variable length**:
 - text (words / characters), speech, audio,
 - time series (stock prices, sensors, weather),
 - DNA / protein sequences, music.

Example: “*The clouds are in the __*”. To predict “sky” you must remember the earlier words. An MLP with a fixed window either ignores context or wastes parameters padding it.

The RNN idea: a loop that carries memory

- Introduce a **hidden state** $\mathbf{h}_t$ that summarizes everything seen up to step t .
- Process the sequence one step at a time, feeding the previous state back in:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}).$$

- The *same* weights are reused at every time step — **parameter sharing across time**.

	Feedforward (MLP)	Recurrent (RNN)
Input length	fixed	variable
Memory of past	none	hidden state $\mathbf{h}_t$
Weights	one set per layer	one set reused every step
Natural data	tabular / images	sequences

Why parameter sharing matters

- Without sharing, a length- T sequence would need T separate weight matrices — parameters grow with length and cannot generalize to unseen lengths.
- Sharing one matrix across all steps means the network learns a *rule* (“how to update memory given a new symbol”) rather than position-specific weights.
- A pattern learned at position 3 is instantly usable at position 300 — this is the temporal analogue of how a CNN shares filters across space.

Payoff: one compact model handles sequences of *any* length and reuses knowledge everywhere along the sequence.

Vanilla (Elman) RNN — the equations

At each time step t , given input $\mathbf{x}_t \in \mathbb{R}^d$ and previous state $\mathbf{h}_{t-1} \in \mathbb{R}^H$:

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

update the hidden state

$$\mathbf{o}_t = W_y \mathbf{h}_t + \mathbf{b}_y$$

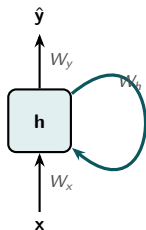
raw scores (logits)

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t)$$

output distribution (if classifying)

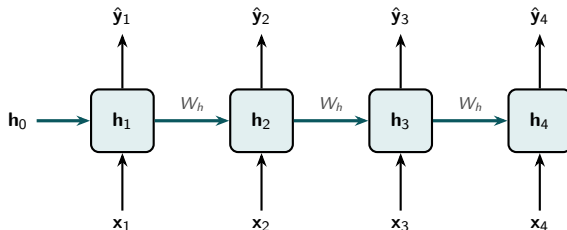
Shapes: $W_x \in \mathbb{R}^{H \times d}$ (input→hidden), $W_h \in \mathbb{R}^{H \times H}$ (hidden→hidden, the “recurrent” matrix), $W_y \in \mathbb{R}^{o \times H}$ (hidden→output). The initial state is usually $\mathbf{h}_0 = \mathbf{0}$.

Folded vs. unrolled view



folded (a loop)

$\equiv$



unrolled through time (same weights repeat)

Green arrows carry the hidden state through time. “Unrolling” turns the loop into a deep feedforward net that shares W_x, W_h, W_y at every step. Training uses **Backpropagation Through Time (BPTT)**.

Worked example — one RNN step (scalar)

Setup ($d=1, H=1, o=1$): $x_1 = 1.0$, $x_2 = 0.5$, $h_0 = 0.0$. Weights: $W_x = 0.8$, $W_h = 0.5$, $b_h = 0.1$, $W_y = 1.2$, $b_y = 0.0$.

$$h_1 = \tanh(0.8 \cdot 1.0 + 0.5 \cdot 0.0 + 0.1) = \tanh(0.9) = \mathbf{0.716}$$

$$\hat{y}_1 = 1.2 \cdot 0.716 = \mathbf{0.859}$$

$$h_2 = \tanh(0.8 \cdot 0.5 + 0.5 \cdot 0.716 + 0.1) = \tanh(0.858) = \mathbf{0.695}$$

$$\hat{y}_2 = 1.2 \cdot 0.695 = \mathbf{0.834}$$

Notice h_2 depends on h_1 , which depends on x_1 : the network *remembers* the first input while processing the second. That recurrence is the whole point.

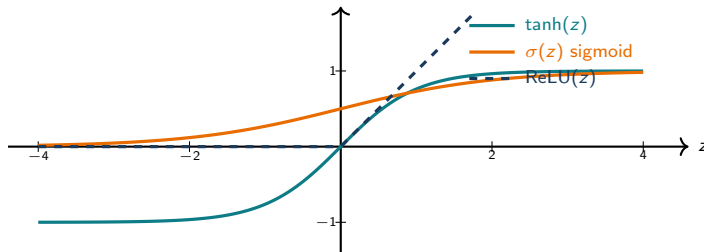
The catch: vanishing / exploding gradients

- BPTT multiplies gradients by W_h (and $\tanh'$) *once per time step*.
- Over a long sequence this repeated product behaves like W_h^T :
 - if the relevant eigenvalue < 1 : gradient $\rightarrow 0$ (**vanishing**) — no long-range learning;
 - if > 1 : gradient $\rightarrow \infty$ (**exploding**) — unstable training.
- Practical fixes: gradient clipping (for exploding), careful init, and — most importantly — **gated cells (LSTM / GRU)** for the vanishing case.

This weakness is exactly why LSTM and GRU were invented; the vanilla RNN is still the clearest model for understanding the mechanics.

Why a nonlinearity at all?

- Without a nonlinearity, stacking/looping linear maps stays linear: $W_a(W_b\mathbf{x}) = W\mathbf{x}$. The whole RNN would collapse to a single linear function — unable to model complex sequences.
- The nonlinearity $\tanh$ lets each step apply a rich, bounded transformation, so depth-in-time actually adds expressive power.



Why tanh and not sigmoid / ReLU (for the state)

Function	Range	Zero-centered?	Effect on the recurrent state
tanh	$(-1, 1)$	yes	Bounded <i>and</i> centered: states stay stable, gradients well-behaved.
Sigmoid σ	$(0, 1)$	no	Always positive $\Rightarrow$ state bi-ased/drifts; smaller gradients ($\max \sigma' = 0.25$).
ReLU	$[0, \infty)$	no	Unbounded $\Rightarrow$ state can blow up when reused each step; positive-only.

Key reasons tanh

wins for the hidden state:

- **Bounded** in $(-1, 1)$: prevents the repeatedly-fed-back state from exploding.
- **Zero-centered**: positive and negative activations $\Rightarrow$ faster, more stable optimization.
- **Stronger gradient** near 0 than sigmoid ($\tanh'(0) = 1$ vs. $\sigma'(0) = 0.25$).

(Sigmoid is still used *inside gates*, where a $[0, 1]$ “valve” value is exactly what we want.)

Parameter count of a vanilla RNN

Let d = input size, H = hidden size, o = output size.

$$\underbrace{H d}_{W_x} + \underbrace{H H}_{W_h} + \underbrace{H}_{\mathbf{b}_h} = \boxed{H(d + H + 1)} \quad (\text{recurrent core})$$

Add the output layer $W_y, \mathbf{b}_y$:

$$\underbrace{o H}_{W_y} + \underbrace{o}_{\mathbf{b}_y} = o(H + 1).$$

Crucial point: the count does *not* depend on the sequence length T — the same matrices are reused at every step. A length-10 and a length-10,000 sequence use identical parameters.

Parameter counting — worked example

Take $d = 3$, $H = 5$, $o = 2$.

$$W_x : H \cdot d = 5 \cdot 3 = 15$$

$$W_h : H \cdot H = 5 \cdot 5 = 25$$

$$\mathbf{b}_h : H = 5$$

$$\text{recurrent core} = H(d + H + 1) = 5(3 + 5 + 1) = \mathbf{45}$$

$$W_y : o \cdot H = 2 \cdot 5 = 10$$

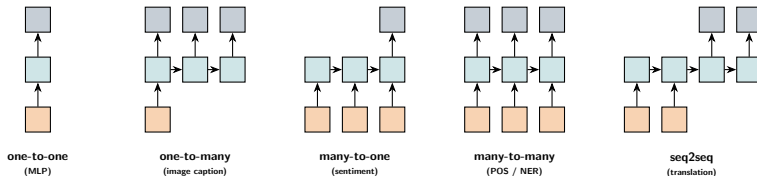
$$\mathbf{b}_y : o = 2$$

$$\text{output layer} = o(H + 1) = 2(5 + 1) = \mathbf{12}$$

$$\mathbf{total} = 45 + 12 = \mathbf{57}$$

As a sanity check for gated cells: an LSTM with the same d, H has 4 such weight groups, GRU has 3 — so LSTM core $= 4H(d + H + 1)$, GRU core $= 3H(d + H + 1)$.

Different RNN configurations (by input/output shape)



Orange = input, **green** = hidden/recurrent, **navy** = output. The recurrence (horizontal green arrows) is shared; only the *alignment* of inputs to outputs changes.

RNN types — when to use which

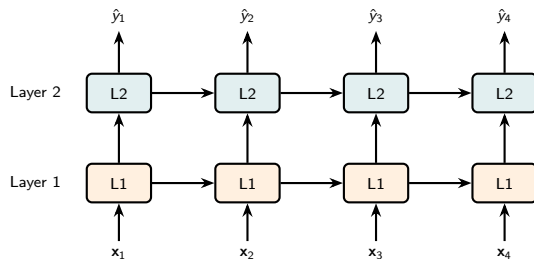
Type	Input $\rightarrow$ Output	Example task
One-to-one	$1 \rightarrow 1$	Plain classification (degenerate RNN = MLP).
One-to-many	$1 \rightarrow \text{many}$	Image captioning; music generation from a seed.
Many-to-one	$\text{many} \rightarrow 1$	Sentiment analysis; sequence classification.
Many-to-many (aligned)	$T \rightarrow T$	POS tagging, named-entity recognition, frame labeling.
Many-to-many (seq2seq)	$T_{\text{in}} \rightarrow T_{\text{out}}$	Machine translation, summarization, speech-to-text.

Also orthogonal:

bidirectional RNNs read the sequence forward *and* backward (only when the whole sequence is available), giving each step past *and* future context.

Stacked (deep) RNNs — why and how

- A single recurrent layer captures one level of temporal features.
- **Stacking** feeds the hidden-state *sequence* of layer ℓ as the input sequence to layer $\ell+1$.
- Lower layers learn short/local patterns; higher layers learn longer, more abstract structure.



Horizontal arrows = recurrence (time); vertical arrows = depth (layer to layer).

Counting parameters in a stacked RNN

Layer 1 sees input size d ; every layer above sees input size H (the width below it).

$$\text{Layer 1} = H(d + H + 1), \quad \text{Layer } \ell \geq 2 = H(H + H + 1) = H(2H + 1).$$

Example: 3-layer RNN, $d = 3$, $H = 5$.

$$\text{Layer 1} = 5(3 + 5 + 1) = \mathbf{45}$$

$$\text{Layer 2} = 5(2 \cdot 5 + 1) = 5 \cdot 11 = \mathbf{55}$$

$$\text{Layer 3} = 5(2 \cdot 5 + 1) = \mathbf{55}$$

$$\text{recurrent total} = 45 + 55 + 55 = \mathbf{155} \text{ (before the output head)}$$

Still independent of sequence length T . Typical depth is 2–4 layers, usually with dropout between layers.

Language modeling: the chain rule of probability

- Goal: model the probability of a sequence $\mathbf{x} = (x_1, \dots, x_T)$ of tokens.
- Factor it *exactly* with the chain rule — one token at a time:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1}).$$

- An RNN implements each factor: the hidden state $\mathbf{h}_{t-1}$ is a fixed-size summary of the entire history $x_{<t}$, so

$$p(x_t \mid x_{<t}) \approx p(x_t \mid \mathbf{h}_{t-1}).$$

This is the **autoregressive** view: predict the next symbol, then condition on it to predict the one after.

From hidden state to next-token distribution

At step t the network turns its state into a probability over the vocabulary V :

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

update state

$$\mathbf{o}_t = W_y \mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^{|V|}$$

logits, one per token

$$p(x_{t+1} = k \mid x_{\leq t}) = \text{softmax}(\mathbf{o}_t)_k = \frac{e^{o_{t,k}}}{\sum_{j=1}^{|V|} e^{o_{t,j}}}$$

normalize to a distribution

Softmax guarantees the outputs are non-negative and sum to 1. The input $\mathbf{x}_t$ is the embedding (or one-hot) of the previous token.

Training: teacher forcing + cross-entropy

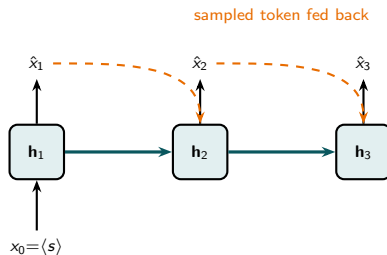
- During training we know the true next token x_{t+1} , so we feed the *true* token as the next input — **teacher forcing**.
- Minimize the negative log-likelihood (cross-entropy) of the whole corpus:

$$\mathcal{L} = - \sum_{t=1}^T \log p(x_{t+1} \mid x_{\leq t}) = - \sum_{t=1}^T \log \text{softmax}(\mathbf{o}_t)_{x_{t+1}}.$$

- Perplexity, the usual metric, is just $\exp(\mathcal{L}/T)$: the model's average “branching factor”.

Lower cross-entropy $\Rightarrow$ the model assigns higher probability to the actual next tokens.

Generation: sample, feed back, repeat



- 1 Start from a seed / start token; compute $\mathbf{h}_t$ and the distribution $p(\cdot \mid x_{\leq t})$.
- 2 **Pick** the next token: greedy (argmax), or *sample* from the distribution.
- 3 Feed that token back as the next input (**orange, dashed**); repeat until an end token or length limit.

Decoding strategies (how to pick the next token)

Strategy	Rule	Behavior
Greedy	$\hat{x}_{t+1} = \arg \max_k p_k$	Deterministic, safe, repetitive.
Temperature	sample from $\text{softmax}(\mathbf{o}_t/\tau)$	$\tau < 1$ sharper/safer, $\tau > 1$ more random/creative. Temperature
Top- k	sample only from k most likely tokens	Cuts the unlikely tail.
Top- p (nucleus)	smallest set with cumulative prob $\geq p$	Adaptive-size candidate set.
Beam search	keep B best partial sequences	Higher-likelihood full sequences.

scaling: $p_k \propto \exp(o_k/\tau)$. As $\tau \rightarrow 0$ it becomes greedy; as $\tau \rightarrow \infty$ it becomes uniform.

Worked example — one generation step

Vocabulary $V = \{a, b, c\}$; suppose the logits are $\mathbf{o}_t = (2.0, 1.0, 0.1)$.

Softmax ($\tau = 1$): exponentials $e^{2.0}=7.39$, $e^{1.0}=2.72$, $e^{0.1}=1.11$; sum = 11.22.

$$p = \left(\frac{7.39}{11.22}, \frac{2.72}{11.22}, \frac{1.11}{11.22} \right) = (0.659, 0.242, 0.099).$$

Greedy picks a. **Sampling** picks a $\approx 66\%$ of the time.

Hotter ($\tau = 2$): use $\mathbf{o}_t/2 = (1.0, 0.5, 0.05)$; $p = (0.484, 0.294, 0.222)$ — flatter, more diverse.

Colder ($\tau = 0.5$): use $\mathbf{o}_t/0.5 = (4.0, 2.0, 0.2)$; $p = (0.842, 0.114, 0.044)$ — sharper, safer. Same model, same state — temperature alone trades off *diversity* vs. *confidence*.

Where RNNs are used

Domain	Tasks
Natural language	language modeling, text generation, translation, sentiment, NER/POS tagging
Speech & audio	speech recognition, speech synthesis, keyword spotting, music generation
Time series	forecasting (finance, weather, demand), anomaly detection in sensor streams
Vision + sequence	video activity recognition, image captioning (CNN encoder → RNN decoder)
Bioinformatics	DNA / protein sequence modeling and labeling

Today Transformers

dominate large-scale NLP, but RNNs (esp. LSTM/GRU) remain strong for streaming, on-device, and low-resource sequential tasks — and are the foundation for understanding all of them.

Summary

- **Why:** sequences have variable length and need memory; MLPs have neither. RNNs add a recurrent hidden state and *share weights across time*.
- **Vanilla RNN:** $\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h)$; unroll to train with BPTT.
- **tanh:** bounded and zero-centered $\Rightarrow$ stable, well-scaled recurrent state; sigmoid is reserved for gate valves, ReLU can explode when reused.
- **Parameters:** core = $H(d + H + 1)$, independent of length; stacked layers add $H(2H + 1)$ each.
- **Types:** one/many $\rightarrow$ one/many, seq2seq, bidirectional, stacked.
- **Generation:** chain rule $\rightarrow$ per-step softmax $\rightarrow$ sample/greedy/temperature $\rightarrow$ feed back.